Cloud-native	Use cloud-native technologies of our provider of choice.	It enables the rapid development of solutions without worrying about capabilities, like hosting, that are not the core of our business.	Prefer open tooling for deployment suitable for our cloud provider. Prefer platform-as-a-service and function-as-a-service solutions.
Mature technologies only	Don't use immature technologies or frameworks unless necessary.	Our products and systems are critical, and once developed are planned to serve and be maintained for multiple years. We value stability more than rapid development.	Choose frameworks and libraries that have more than 1,000 stars on GitHub. Use a framework or library only after first applying a minor/patch release with fixes.
Automate everything	All manual work should be automated whenever possible.	Manual work, like deployments, is always more error prone and less repeatable than automated work.	Deployments from development to production environments should be fully automated using pipeline as a code. All tests should be automated.

9.2.2 Capturing architectural drivers for a data-product design

In section 9.2.1, we described what we mean by architectural drivers with simple examples. This section focuses on capturing architectural drivers for our data products, using the example of a Cost Statement data product, which is a merge of cost statements from preproduction, production, and postproduction. Cost statements are about spending during the whole process of production.

ANALYZE FUNCTIONAL REQUIREMENTS

Let's imagine you are the software architect responsible for designing a newly established data product. The data product owner already knows the data scope of the product and has already interviewed all possible users, and